auth0-mcp-demo: what we built

Status as of 2026-10-21. Repository: github.com/ExpertVagabond/auth0-mcp-demo (private, main at dfd4eed). Tenant: dev-pimpusbqkepr5gdl.us.auth0.com (development environment, trial plan). This document contains no secrets, client IDs, or tokens.

1. Summary

We built an MCP (Model Context Protocol) server that requires OAuth access tokens from Auth0 and enforces permissions on each tool. The server acts for the signed-in user, not as itself. It exchanges the user's token for one addressed to a backend reports API. It also reads a user's linked GitHub account through Auth0 Token Vault. It is a reference implementation for a developer-advocate role at Okta/Auth0: built, tested locally, and verified live against a real tenant. It is not running in production.

The main use case is a personal assistant. An agent acts for one person across that person's own services, and each tool is limited to the permissions that person granted.

2. Components

Component	What it does
`server.mjs`	MCP server on streamable HTTP at `/mcp`. Validates bearer tokens against the issuer's JWKS, checks issuer, audience, and expiry. Serves protected-resource metadata (RFC 9728). Enforces per-tool scopes with HTTP 403. Performs RFC 8693 token exchanges, cached until shortly before expiry. Caps request bodies at 1 MB and returns JSON-RPC parse errors as 400.
`reports-api.mjs`	Backend reports API. `GET /reports/{week}`. Accepts only tokens whose audience is the backend API and that carry the `read:reports` permission. Returns 401 for a missing or wrong-audience token, 403 for a missing permission, and 404 for an unknown week. The data is sample data.
`scripts/login-check.mjs`	Browser login with authorization code and PKCE. Sends the RFC 8707 `resource` parameter by default. Can target a connection (for example GitHub) and a custom scope. Saves the token to a private file and never prints it.
`scripts/live-check.mjs`	Calls the running server with a saved token: tool list, `whoami`, `read_report`, `github_profile`, `github_open_prs`.
`scripts/connect-github.mjs`	Links a GitHub account to the signed-in user for Token Vault (Connected Accounts), using the My Account API.
`log-receiver/`	Cloudflare Worker that receives Auth0 log stream batches. Requires a bearer token. Logs only event counts and types. Stores nothing.
`test/e2e.test.mjs`	22 local tests. A mock issuer signs real RS256 tokens and runs token exchange (including the Token Vault exchange). A stub GitHub API stands in for api.github.com.

3. MCP tools

Tool	Input	Required scope	What it returns
`whoami`	none	any valid token	Subject and granted scopes
`read_report`	`week`, format YYYY-Www	`read:reports`	The week's report from the backend reports API, reached through token exchange
`github_profile`	none	`read:github`	The GitHub login linked to the user (the GitHub token is used server-side and never returned)
`github_open_prs`	`limit`, integer 1 to 20 (default 10)	`read:github`	The user's open pull requests, newest first: title, URL, repository, creation date. Public repositories only, because the GitHub connection requests no scopes.

4. How authorization works

- **Token validation:** the server verifies the RS256 signature against the tenant's JWKS, then checks the issuer, the audience (the MCP resource identifier `https://reports.demo.test/`), and expiry. A failed check returns 401 with a `WWW-Authenticate` challenge that names the protected-resource metadata.
- **Permission rule:** a permission counts only when it appears in both the `scope` claim (what the client was granted) and the `permissions` claim (what the user holds). A token without a `scope` claim grants nothing. This is the fail-closed default.
- **Per-tool scopes:** a `tools/call` without the tool's scope gets HTTP 403 with `error="insufficient_scope"` and the scope named. This is the behavior the MCP spec requires.
- **No token passthrough:** the user's token is never forwarded to another API. The server exchanges it (RFC 8693, on-behalf-of) for a token whose audience is the backend API.
- **Token Vault:** for GitHub, the server exchanges the user's token for the GitHub token Auth0 stored for that user. That GitHub token is used for one GitHub API call and never returned to the client.
- **Resource parameter:** clients send the RFC 8707 `resource` parameter. A live login with `resource` produced the same audience claim as `audience`.

5. Auth0 tenant configuration (dev-pimpusbqkepr5gdl)

APIs

- **MCP resource** `https://reports.demo.test/`: RS256, RFC 9068 token profile, RBAC enforced. Scopes `read:reports` and `read:github`.
- **Reports backend** `https://reports-backend.demo.test/`: scope `read:reports`. Skip-consent enabled for first-party clients.
- **Auth0 My Account API** (`/me/`): enabled, with user grants for connected-accounts scopes for the Token Vault app.

Applications

- **Login test app** (native, first-party): used for the browser login checks. Google and GitHub enabled.

- **MCP client app** (native, first-party): the pre-registered client a real MCP client should use. Google and GitHub enabled. Client grant to the MCP resource.
- **Token exchange client** (resource-server type, bound to the MCP resource): on-behalf-of exchange, and the Token Vault federated grant.
- **Token Vault app** (regular web, confidential): Connected Accounts for GitHub, with the My Account API grants.
- **Machine-to-machine test app**: a client-credentials token for the first live check.
- **Dynamically registered test clients**: created during testing and deleted afterward.

Roles, users, connections, Actions

- **Report Reader role:** `read:reports`, `read:github` on the MCP resource. Assigned to the GitHub-linked test user and to a Google test user.
- **Users:** one Google-login user (no GitHub link) and one GitHub-login user with a linked GitHub account. Token Vault connects only the GitHub-login user.
- **Connections:** Google (social), GitHub (social, using Auth0 development keys), Username-Password-Authentication. GitHub has Connected Accounts enabled.
- **GitHub OAuth app:** registered by the developer (ExpertVagabond). Its callback is the tenant's login callback. Its client secret is stored locally, owner-only.
- **Action:** `reports-claims`, post-login. Adds the claim `https://reports.demo.test/email_verified` to access tokens. Deployed and bound to post-login.
- **Organization:** `auth0-mcp-demo`. Created, not used by the build.

Tenant settings

- Resource Parameter Compatibility Profile: on. Include Issuer in Authorization Responses: on. Strict DPoP refresh token binding: on (default).
- One-time-password MFA: enabled. Breached-password detection: enabled (block and user notification). Brute-force protection: on (default).
- Dynamic client registration: **off**. It was turned on briefly for testing and then turned off again. Registered clients are third-party and can't use social connections on this tenant.
- Log stream `reports-receiver`: HTTP type, delivering to the Cloudflare Worker. Delivery verified.
- Not set up: Token Vault for additional services, log storage, and any paid feature. The plan is a trial. Check the Subscription tab after it ends.

6. Verification results

Check	Result
Local test suite (`npm test`)	**22 of 22 pass**
Live check with a machine-to-machine token: `whoami`, `read_report`	**Pass**
Browser login as a Google user and as a GitHub user, with PKCE and the `resource` parameter	**Pass. Token claims include both the `scope` and `permissions` claims.**

On-behalf-of exchange, reports API	**Pass. The reports API accepts the exchanged token (200) and refuses the user's own MCP token (401).**
Token Vault exchange for GitHub, then a call to api.github.com	**Pass. GitHub returned 200 for the linked user.**
`github_profile` and `github_open_prs` live	**Pass. The live call returned the linked login and real open pull requests.**
Assistant test through an MCP client: limits 1, 20, 0; token without `read:github`	**Pass. Limit 0 is rejected by the schema. The token without the scope gets HTTP 403 with `insufficient_scope`.**
MCP Inspector CLI (third-party client): list tools, call each tool with a token	**Pass**
Unauthenticated request	**401 with the metadata challenge. Protected-resource metadata lists the tenant as the authorization server.**
Log stream delivery to the Worker	**Pass. An Auth0 event batch was received.**
Dynamic client registration	Registration succeeds. A registered client can't log in through a social connection (see findings).

7. Findings from building it (friction log, 12 items)

1. Release status for Auth for MCP conflicts across Auth0 pages: one says GA, a search summary of the quickstart said Early Access, and the quickstart page itself gives none.
2. Permissions arrive in a `permissions` claim. A server that reads only `scope` refuses valid tokens.
3. The 401/403 challenge format isn't shown in the Auth0 pages. It was checked against the MCP spec.
4. `resource` and `audience` produce the same audience claim here.
5. Scopes appear in the access token only when RBAC and the token profile are set up.
6. On-behalf-of requires a `resource_server` client bound to the audience of the incoming token. The error message doesn't name the binding.
7. Exchanged tokens carry `permissions` but no `scope` claim.
8. The target API needs "Allow Skipping User Consent for Verifiable First-Party Clients" for the exchange.
9. A role assigned after login applies only to tokens issued afterward.
10. Token Vault for GitHub needs your own GitHub OAuth app. Auth0's development keys stop at the account-linking step.
11. The account-linking step needs a user-subject grant on the My Account API. A client-subject grant isn't enough.
12. Dynamically registered clients are third-party and strict. They can't use social connections, and the mode can't be changed afterward. The workaround is a first-party client created ahead of time, with its client ID given to the MCP client. The MCP spec allows this.

8. Known limits

- This is a dev tenant on a trial plan. It is not production.
- The GitHub connection uses Auth0's development keys. Production use needs the developer's own GitHub OAuth app for the connection.
- `github_open_prs` returns public pull requests only. Private repositories need a broader GitHub scope, which hasn't been granted.
- The reports data is sample data.
- Dynamic client registration is off. Other MCP clients must be pre-registered by an admin before they can sign in.
- The log receiver stores nothing. Retention hasn't been decided.
- The GitHub client secret appeared in a screenshot in chat. It was approved for this demo, but it should be rotated before anything public.

9. How to run it

```
cd projects/auth0-mcp-demo
npm install
npm test                                    # 22 local tests

# Reports API (backend)
AUTH0_ISSUER=https://dev-pimpusbqkepr5gdl.us.auth0.com/ \
REPORTS_AUDIENCE=https://reports-backend.demo.test/ PORT=3100 node reports-api.mjs

# MCP server. Secrets come from the local config folder, not the repo.
AUTH0_ISSUER=https://dev-pimpusbqkepr5gdl.us.auth0.com/ \
MCP_RESOURCE=https://reports.demo.test/ PORT=3000 \
REPORTS_API_URL=http://localhost:3100 \
TOKEN_EXCHANGE_CLIENT_ID=... TOKEN_EXCHANGE_CLIENT_SECRET=... \
REPORTS_AUDIENCE=https://reports-backend.demo.test/ node server.mjs

# Get a token (browser login), then check the server
AUTH_PARAM=resource LOGIN_CONNECTION=github LOGIN_CLIENT_ID=... \
AUTH0_DOMAIN=dev-pimpusbqkepr5gdl.us.auth0.com MCP_RESOURCE=https://reports.demo.test/ \
TOKEN_OUT=... node scripts/login-check.mjs
TOKEN_FILE=... node scripts/live-check.mjs
```

Local secrets live in `~/.config/auth0-mcp-demo` with owner-only permissions. The `.gitignore` excludes credential-style filenames.

10. Open items

- Decide log-receiver retention. The current setting stores nothing.
- Check the Subscription tab after the trial ends, which is about 11/12 based on the 22 days shown on 10/21.
- Edit and decide on publishing the identity draft (`identity-for-mcp-servers-DRAFT.md`). It isn't published.

- Optional next connection: Google Calendar, for a "what's on my day" tool. It needs a Google connection with calendar access and offline access.
- Optional: move the assistant test script into the repo.
- Disk: about 9.9 GiB free on the system volume, just under the 10 GB alert threshold.